A Categorical Formulation of an NLP Pipeline
with k-Fold Cross-Validation

A Functional Programming Approach

Introduction

This document outlines the functional specification for an NLP classification pipeline, framed using the language of category theory. The workflow is defined as a category $\mathcal{W}_{\text{NLP}}$, where data types are **Objects** and functions are **Morphisms**.

A morphism $f : A \to B$ is a pure function that accepts an object of type A and returns a new object of type B. The pipeline uses k-fold cross-validation, necessitating a clear separation between morphisms applied globally (once) and those applied iteratively within each fold.

1 Global Morphisms (Run Once)

These morphisms are applied to the entire dataset *before* any splitting occurs.

`preprocess: Corpus` $\to$ `PreprocessedCorpus`

- **Input (**$Corpus$**):** A list of raw text documents, e.g., `List<String>`.
  Output ($PreprocessedCorpus$**):** A list of tokenized and cleaned documents, e.g., `List<List<String>>`.
  Description: A pure function applying all stateless preprocessing: tokenization, lowercasing, and removal of punctuation and stopwords.

`create_folds: (PreprocessedCorpus` $\times$ `LabelVector` $\times$ k`)` $\to$ `FoldGenerator`

- **Input:** The full $PreprocessedCorpus$, the corresponding $LabelVector$ (e.g., `List<Integer>`), and an integer k.
  Output ($FoldGenerator$**):** An iterator or list of k `Fold` objects. A `Fold` is defined as a product: (TrainIndices $\times$ TestIndices).
  Description: Generates k pairs of index arrays. This process is typically stratified on the $LabelVector$ to ensure class balance within each fold.

2 Fold-Specific Morphisms (Run k Times)

These morphisms are composed and executed k times, once per `Fold` generated by `create_folds`.

`get_fold_data: (PreprocessedCorpus` $\times$ `LabelVector` $\times$ `Fold)` $\rightarrow$ `(TrainData` $\times$ `TestData)`

- **Input:** The complete $PreprocessedCorpus$, $LabelVector$, and one `Fold` object (containing train/test indices).
  Output: A product of two data objects: $TrainData = (Corpus_{\text{train}}, Labels_{\text{train}})$ and $TestData = (Corpus_{\text{test}}, Labels_{\text{test}})$.
  Description: Slices the full dataset based on the indices for the current fold.

`fit_vectorizer: Corpus`$_{\text{train}}$ $\rightarrow$ `TrainedVectorizer`

- **Input ($Corpus_{\text{train}}$):** The list of preprocessed documents for this fold's training set.
  Output ($TrainedVectorizer$): An immutable state object (e.g., a dictionary or hash map) containing the vocabulary and feature-scaling statistics (e.g., IDF weights).
  Description: A critical morphism for preventing data leakage. The vocabulary and statistics are derived **only** from the fold's training data.

`transform: (PreprocessedCorpus` $\times$ `TrainedVectorizer)` $\rightarrow$ `FeatureMatrix`

- **Input:** A corpus (e.g., $Corpus_{\text{train}}$ or $Corpus_{\text{test}}$) and the $TrainedVectorizer$ fitted on this fold's training data.
  Output ($FeatureMatrix$): A numerical representation (e.g., DTM or TF-IDF matrix).
  Description: Applies the vectorization (token-to-index mapping and weighting) defined by the $TrainedVectorizer$.

`train: (FeatureMatrix`$_{\texttt{train}}$ $\times$ `Labels`$_{\texttt{train}}$`) → TrainedModel`

- **Input:** The numerical training matrix and corresponding labels for the current fold.
  Output ($TrainedModel$**):** An immutable state object containing the fitted model parameters (e.g., SVM coefficients, RF trees).
  Description: The core training morphism that fits the classifier to the data.

`predict: (TrainedModel ×` `FeatureMatrix`$_{\texttt{test}}$`) → PredictionVector`

- **Input:** The $TrainedModel$ from this fold and the $FeatureMatrix_{\text{test}}$ from this fold.
  Output ($PredictionVector$**):** A list of predicted labels (e.g., `List<Integer>`).
  Description: Generates predictions for the hold-out test data.

`evaluate_fold: (PredictionVector ×` `Labels`$_{\texttt{test}}$`) → FoldMetrics`

- **Input:** The $PredictionVector$ and the true $Labels_{\text{test}}$ for this fold.
  Output ($FoldMetrics$**):** A data object (e.g., a dictionary) containing performance scores for this single fold (e.g., F1, Accuracy).
  Description: Calculates the model's performance on the hold-out data.

3 Final Aggregation Morphism (Run Once)

This morphism is applied *after* the k-fold loop is complete.

`aggregate_metrics: List<FoldMetrics> → AggregatedMetrics`

- **Input:** A list containing the k $FoldMetrics$ objects, one from each completed fold.
  Output ($AggregatedMetrics$**):** A final summary data object.
  Description: Aggregates the k individual performance scores into a final report, typically by calculating the mean (μ) and standard deviation (σ) for each metric (e.g., $\mu_{\text{F1}}, \sigma_{\text{F1}}$).